

Department of Electrical Engineering
Indian Institute of Technology Kharagpur
Machine Learning for Signal Processing Laboratory
(EE69210)
Spring, 2023-24

Experiment 8:

Fashion MNIST Classification using Fully Connected Networks with Batch Normalisation and Dropouts

Grading Rubric

	Tick the best applicable per row			Points
	Below Expectation	Lacking in Some	Meets all Expectation	
Completeness of the report				
Organization of the report (5 pts)				
Quality of figures (5 pts)				
Dataset Loading and explanation (10 pts)				
Model training, inference, and explanation for the base model with discussions. (25 pts)				
Model training, inference, and explanation for the model with dropouts with discussions. (25 pts)				
Model training, inference, and explanation for the model with batch normalization with discussions. (25 pts)				
Discussion on how to improve accuracy. (5 pts)				
TOTAL (100 pts)				

1 Code of Conduct

Students are expected to behave ethically both in and out of the lab. Unethical behaviour includes, but is not limited to, the following:

- Possession of another person's laboratory solutions from the current or previous years.
- Reference to, or use of another person's laboratory solutions from the current or previous years.
- Submission of work that is not done by your laboratory group.
- Allowing another person to copy your laboratory solutions or work.
- Cheating on quizzes.

The rules of laboratory ethics are designed to facilitate these goals. We emphasize that laboratory TAs are available to help the student understand the basic concepts and answer the questions asked in the laboratory exercises. By performing the laboratories independently, students will likely learn more and improve their performance in the course as a whole. Please note that it is the student's responsibility to ensure that the content of their graded laboratories is not distributed to other students. If there is any question as to whether a given action might be considered unethical, please see the professor or the TA before you engage in such actions.

2 Fashion-MNIST Classification using Fully Connected Neural Network

The Fashion-MNIST classification task involves employing a fully connected network architecture with dropout layers to recognize different fashion items in the Fashion-MNIST dataset. The dataset comprises 60,000 labeled images in the training set, where each 28x28 pixel grayscale image represents a specific fashion item. The architecture incorporates linear layers with dropout for regularization, making it suitable for capturing features in the Fashion-MNIST images.

The model is trained on the training set and then evaluated on a separate test set containing 10,000 images to assess its accuracy and generalization performance in classifying various fashion items. The loss function used for training is negative log-likelihood, which is appropriate for multi-class classification tasks.

2.1 Dataset Description

The **Fashion-MNIST** dataset is a prominent benchmark in the field of machine learning and computer vision, serving as an alternative to the traditional MNIST dataset. It features 28×28 pixel grayscale images representing various fashion items, with each image associated with a label indicating the specific category of the depicted item (e.g., T-shirt, trouser, pullover, etc.).

Key characteristics of the Fashion-MNIST dataset include:

1. **Image Size:** Each image is a 28×28 pixel square, resulting in a total of 784 pixels per image.
2. **Grayscale:** Similar to MNIST, the images are monochromatic, with pixel intensity values ranging from 0 (black) to 255 (white).
3. **Categories:** Fashion-MNIST comprises 10 categories representing various fashion items, including T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boots. There are 7,000 images from each category thereby resulting in a total of 70,000 images

4. **Purpose:** Fashion-MNIST is widely utilized as an introductory dataset for training and evaluating image classification algorithms, providing a diverse set of categories beyond handwritten digits. It serves as a valuable tool for assessing the performance of machine learning models, particularly in the context of deep learning and neural networks.
5. **Challenges:** Despite its seemingly straightforward nature, Fashion-MNIST presents challenges due to variations in clothing styles, textures, and the relatively small image size. These challenges make it a crucial dataset for testing and refining models designed for fashion item recognition.

2.2 Architecture of the Fully Connected network

We have a Fully Connected neural network. It has five layers including one output layer.

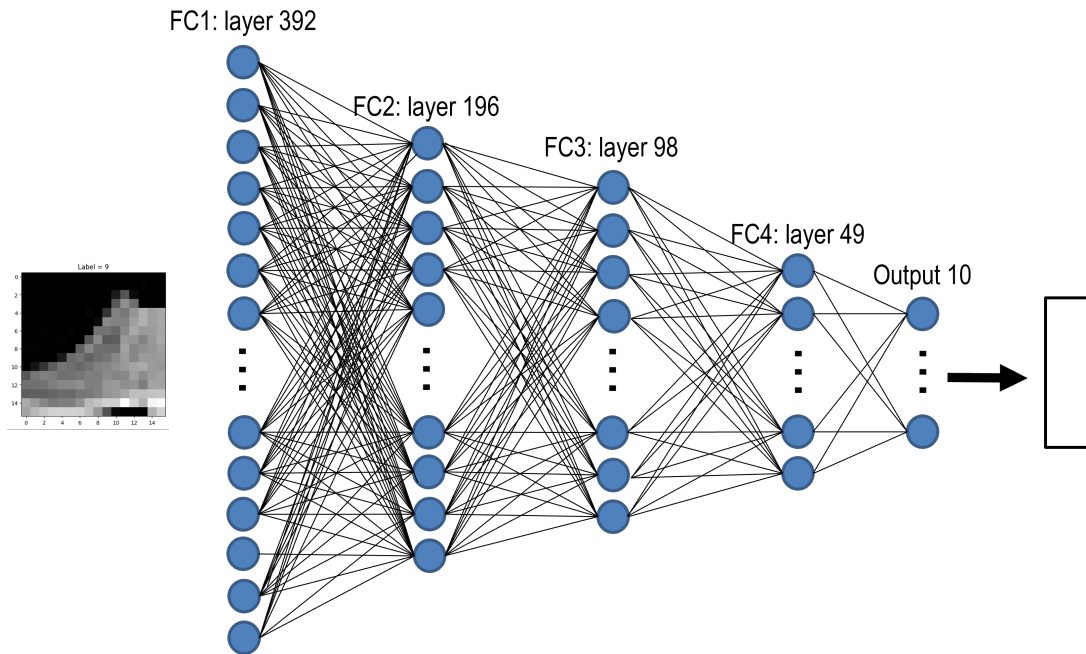


Figure 1: The neural network for this lab manual features four fully connected layers, denoted as FC, with the number following each FC indicating the number of outputs from that layer. The final output layer consists of 10 neurons, each corresponding to a specific class of the dataset, providing a structured representation for classification.

1. FC1 :

- **Fully Connected :** This is the first layer of the network. being a fully connected layer, it takes in the flattened image as input and has 392 output neurons.
- **Non-Linearity :** The layer is followed by ReLU activation.

2. FC2 :

- **Fully Connected :** Fully connected layer with 196 output neurons.
- **Non-Linearity :** The layer is followed by ReLU activation.

3. FC3 :

- **Fully Connected :** Fully connected layer 98 output neurons.

- **Non-Linearity** : The layer is followed by ReLU activation.

4. FC4 :

- **Fully Connected** : Fully connected layer with 49 output neurons.
- **Non-Linearity** : The layer is followed by ReLU activation.

5. Output :

- **Fully Connected** : This is the final and output layer of the network. It has 10 output neurons for each class of the dataset.
- **Non-Linearity** : Softmax activation for multiclass classification

2.3 Loss function

Cross-entropy loss, commonly used in machine learning for classification tasks, quantifies the difference between predicted probabilities and actual class labels. Specifically, in the context of a neural network, the cross-entropy loss measures the information gained with the model's predictions. The formula takes the negative logarithm of the predicted probability assigned to the correct class. A lower cross-entropy loss indicates that the model's predictions align well with the actual labels, while a higher loss suggests a greater divergence. Minimizing cross-entropy is a common objective during training as it encourages the neural network to converge towards accurate predictions. Mathematically, cross-entropy loss is given by

$$L(y, \hat{y}) = - \sum_{i=1}^C y_i \cdot \log(\hat{y}_i) \quad (1)$$

where y is the ground truth label vector and $\hat{y}$ represents the predicted probability vector.

2.4 Learning Rate

The learning rate is a crucial hyperparameter determining the step size at which the model adjusts its weights during training. We will put the learning rate as 0.001, implying a moderate step size, allowing the model to adjust its parameters gradually. This balance is essential for avoiding overshooting or convergence issues. Experimenting with different learning rates can impact the training dynamics, affecting the model's ability to converge to an optimal solution.

2.5 Dropouts and Batch Normalization

1. **Dropout** : During training, the output of each neuron is set to zero with a probability p . This is mathematically represented as:

$$Output = Input \times Bernoulli(p) \quad (2)$$

where $Bernoulli(p)$ is a random binary mask with a probability p .

2. **Batch Normalization** : For a given layer, the Batch Normalization process involves normalizing the inputs (x) over the mini-batch, followed by scaling and shifting using learnable parameters (γ and β). The normalized output ($\hat{x}$) is computed as:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad (3)$$

where μ is the mean, σ is the standard deviation, and ϵ is a small constant to avoid division by zero. The scaled and shifted output is then obtained as:

$$Output = \gamma \hat{x} + \beta \quad (4)$$

Here, γ and β are parameters learned during training to allow the network to adapt the normalization to the data.

3 Experiments on Fashion-MNIST digit classification

3.1 Importing essential libraries

```
1 #importing libraries
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from torchvision import datasets, transforms
7 import matplotlib.pyplot as plt
8 from collections import OrderedDict
```

Listing 1: Code to import essential libraries

3.1.1 Download and preparation of the dataset in train, test and validation. Creating the dataloaders.

```
1 # Download training and testing data
2 transform = transforms.Compose([transforms.ToTensor()])
3 train_dataset = datasets.FashionMNIST('F_MNIST_data', download=True, train=True,
4   , transform=transform)
5 test_dataset = datasets.FashionMNIST('F_MNIST_data', download=True, train=False,
6   , transform=transform)
7 # split train set into training (80%) and validation set (20%)
8 train_len = len(train_dataset)
9 indices = list(range(train_len))
10 np.random.shuffle(indices)
11 split = int(np.floor(0.2 * train_len))
12 val_ids, train_ids = indices[:split], indices[split:]
13 # prepare dataloaders
14 train_sampler = torch.utils.data.sampler.SubsetRandomSampler(train_ids)
15 train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=128,
16   , sampler=train_sampler)
17 val_sampler = torch.utils.data.sampler.SubsetRandomSampler(val_ids)
18 val_loader = torch.utils.data.DataLoader(train_dataset, batch_size=128, sampler
19   =val_sampler)
20 test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=128, shuffle
21   =True)
```

Listing 2: Code to download the dataset

3.1.2 Visualisation of the dataset.

```
1 # Size of train and test datasets
2 print("Length of train data is "+str(len(train_sampler)))
3 print("Length of test data is "+str(len(test_dataset)))
4 print("Length of validation data is "+str(len(val_sampler)))
5 image, label = next(iter(train_loader))
6 print(image[0].shape, label.shape)
7 desc = ['T-shirt/top', 'Trouser', 'Pullover', 'Dress', 'Coat', 'Sandal', 'Shirt', 'Sneaker', 'Bag', 'Ankle Boot']
8 print(desc[label[0].item()])
9 plt.imshow(image[0].numpy().squeeze(), cmap='gray');
```

Listing 3: Code to visualise the dataset

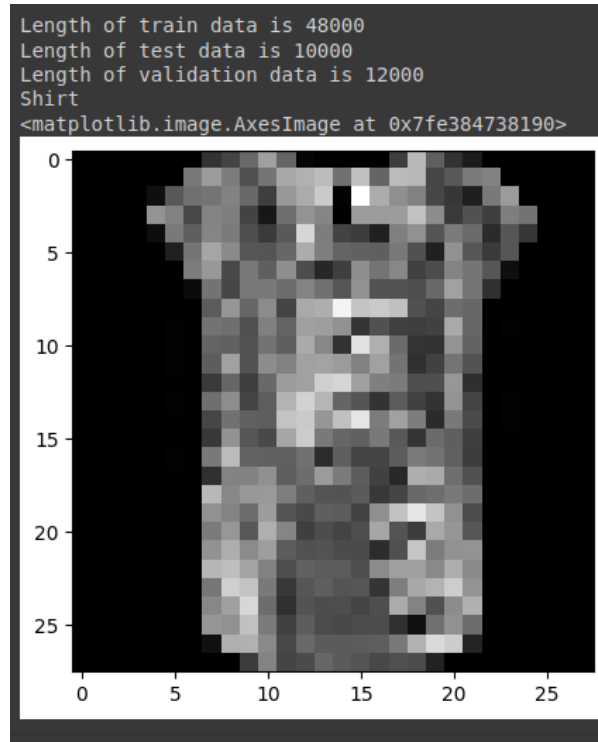


Figure 2: The first image and label of the training data.

3.1.3 Assignments to solve and report

Write the codes for the following tasks in a .ipynb file, including all visualizations and submit the executed file. Submit also a separate pdf of your report on this experiment. This report should describe your observations and reasoning while executing these experiments.

1. Rerun all the steps discussed till now, rerunning all the codes. Check your consistency in generating data similar to those presented earlier.
2. Create the model architecture.
3. Show the size of every tensor in the forward pass.
4. Train it for 50 epochs with a learning rate of 0.0001
5. Plot the training loss and accuracy, checking it with (3).
6. Evaluate the model and show the testing accuracy.
7. Rerun the experiments with introducing Dropouts with a probability of 0.25 after ReLU activation in FC1, FC2, FC3, and FC4 and check how the performance varies from the without dropouts.
8. Rerun the experiments with introducing Batch Normalisation after every linear layer in FC1, FC2, FC3, and FC4 and check how the performance varies from the without Batch Normalisation.
9. Discuss how the testing accuracy can be improved. (Hint - Incorporating spatial information)

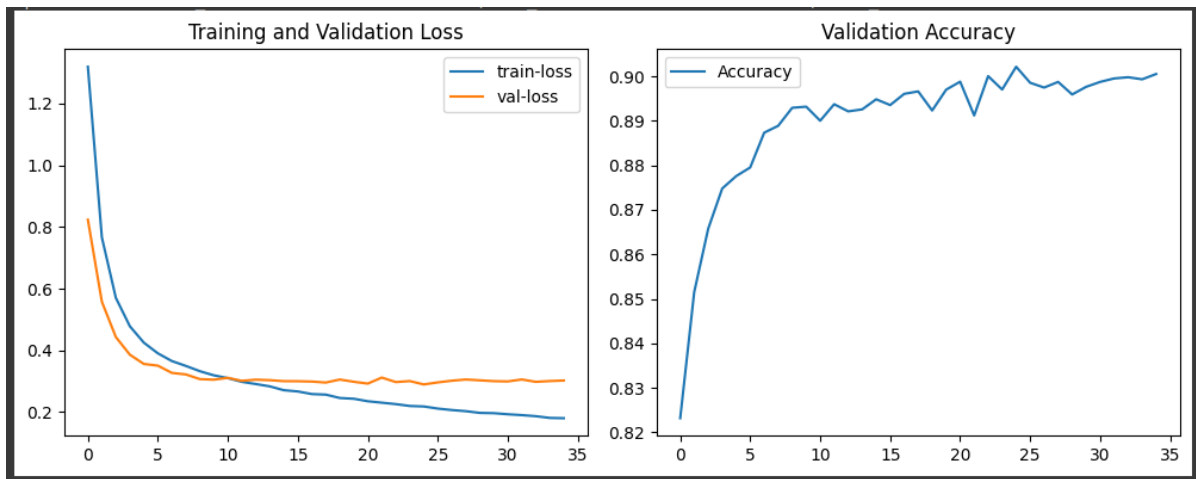


Figure 3: Training loss and accuracy.